

HYDROOPTIMIZE: Automatically Optimize Distributed Protocols by Manipulating Time

ACM Reference Format:

. 2026. HYDROOPTIMIZE: Automatically Optimize Distributed Protocols by Manipulating Time. In . ACM, New York, NY, USA, 3 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Outline

1. Intro

- a. Distributed protocols are the core of cloud computing.
- b. They are difficult to optimize correctly due to their complexity. This has 3 downsides: (1) industry opts for simple, suboptimal protocols, (2) the iteration cycle is long because redesigns require new proofs, and (3) it is difficult to combine optimizations.
- c. Existing works present optimizations but their performance is limited (show Paxos throughput upper bound) because certain nodes cannot be scaled due to their highly synchronous logic.
- d. Our key insight is that logic is often unnecessarily synchronous, resulting in coincidental joins, co-batching, and sequentiality, and that the key to powerful optimizations is distinguishing between necessary and coincidental synchrony.
- e. We present HYDROOPTIMIZE, a system that automatically optimizes distributed protocols. Programmers write simple protocols, and the optimizer combines known approaches from prior work to find the optimal set of rewrites.
- f. We generalize the optimizations introduced by Mencius and Scalog by exploiting coincidental joins.
- g. We generalize the different decoupling techniques by exploiting coincidental co-batching and sequentiality.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org. *Conference'17, Washington, DC, USA*

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

h. Evaluation

- i. Limitations (liveness, fault tolerance, general omission model, same as before)

2. Background

- a. Existing optimizations
- b. Theme across optimizations: Show that given the same inputs that arrive at the same times on the new and original protocols, the new protocol will produce some output that could've been produced before.
- c. Proof technique (observable equivalence, linearizability)

3. The Unnecessarily Strict Semantics of Time

- a. Coincidental joins: The rewrites ensured that values that (1) arrive at the same tick and (2) would have joined in that tick will also join in the new protocol, even though their co-arrival is coincidental. Existing rewrites require that possible joins are not altered (partitioning with co-hashing, partitioning with dependencies).
- b. Coincidental co-batching: For each tick, different aggregations and negations must operate on the same sets of data, whether preserving the batching is necessary for correctness or not. This means that existing rewrites either do not work if there is negation or aggregation (functional, monotonic decoupling) or require waiting on the same batches (state machine decoupling).
- c. Coincidental sequentiality: (Non-monotonic) rules that execute in the next must witness (and be blocked on the deduction of) values derived in the previous tick, whether actually seeing that previous value is necessary for correctness or not. Existing rewrites either avoid decoupling or require coordination (asymmetric decoupling).

- d. Overcoming the Limitations of Time: (introduce later sections)

4. Exploiting coincidental joins with Time partitioning

- a. Overview (start with stateless, derive time partitioning, show that Mencius and Scalog are specific implementations of time partitioning)

- b. Stateless partitioning: Partitioning (as defined in SIGMOD) required co-hashing on specific attributes because facts with the same values for those attributes join. The facts that would join are: newly arrived facts (or newly derived facts), and persisted facts. If we know that there are no persisted facts, then there exists a run where no 2 facts coincide (and thus they do not join) and we do not need to co-hash on attributes. This isn't actually a useful rewrite, since it would only apply to a program that is trying to check the (random) co-existence of other inputs.
 - c. Time partitioning: If there *are* persisted facts, but those persisted facts do not depend on the *values* of prior facts (they may depend on their *cardinality*), then each partition can simulate the execution of prior ticks to derive the persisted state at any tick. Combined with stateless partitioning, this means that each partition can be responsible for a subset of the ticks and derive the system state at that tick. However, naively time partitioning introduces 2 problems:
 - d. (1) Ticks that never arrive: A partition may execute a future tick anticipating that some prior number of ticks will execute (on some set of inputs). Those inputs, however, may never arrive. Consider a client that sends a finite number of messages.
 - e. (2) Linearizability: A client may send a message to a partition with a higher tick, receive the output, then send a causally dependent message to a partition with a lower tick, breaking causality (and linearizability).
 - f. Mencius: It time partitions the Paxos leader and permits breaks in linearizability. Although this breaks Paxos semantics locally, it is actually observably equivalent to a run of Paxos where the leader switches and only its later slots are committed. Our optimizations are local and do not understand the intricacies of leader election. We need a way to make sure that a partition that executes a future tick does so only after knowing that all prior ticks will not receive additional messages. The challenge is doing so without essentially processing one tick at a time (which would be the same as not partitioning).
 - g. Pipelined time partitioning: Each partition tells the next partition the cardinality of its message batch before processing the batch. Although partitions have to wait until they get this message, they execute their batches in parallel.
 - h. Broadcast time partitioning: Each partition broadcasts the cardinality of its message batch to all other partitions. A partition that has received the cardinality messages from all other partitions can execute its batch.
- This is a tradeoff between wait time and message complexity.
- i. Coordinated time partitioning: Similar to quorum certificates in PBFT, a natural extension of broadcast time partitioning is to have each partition send their cardinalities to a coordinator that broadcasts to each partition. This is (roughly) the approach that Scalog takes.
 - j. Limitations. Mencius and Scalog rely on optimizations specific to consensus; Mencius relies on weakening consistency and Scalog relies on background replication. Our local optimizations are more general but also less performant because they have to maintain linearizability and cannot optimistically replicate (acceptor partitioning is based on slot).
5. Removing coincidental co-batching and sequentiality with HYDRO
 - a. Slices (or ticks?)
 - b. Show code snippet
 - c. Subsuming existing decoupling techniques: Show that functional, monotonic, state machine decoupling can all be accomplished (naturally) with slices.
 6. Implementation
 - a. Optimizer rewrites Hydro IR
 - b. Decoupling = inserting a Network node. Describe constraints that must be held
 - c. Partitioning = modifying inputs to Network nodes.
 - d. Naive optimization (decouple and partition as much as possible, then fuse) is suboptimal, show example. There is no optimal order. Decoupling and partitioning are intertwined.
 - e. Encoding decoupling and partitioning constraints in ILP.
 7. Evaluation
 - a. General efficacy: Show results on Paxos (with replicas), Paxos (no replicas), distributed CAS, AWS chain replication, AWS strongly consistent secondary indices, Apache Flink workloads, maybe lobsters or websubmit. Show different distribution of reads/writes for CAS and strongly consistent indices. Discuss optimizations that were applied to each one, and how long it took to arrive at the optimal protocol. Show numbers from our prior work on Paxos.
 - b. Case study: AWS chain replication. (For all following sections, just show breakdown of chain replication, but still briefly discuss other protocols and why certain techniques are more or less helpful for them.)

- c. Time partitioning: Show performance of different time partitioning mechanisms.
- d. General decoupling: Show that staleness is ok between learning that a value has been committed and leader re-election (and updating the local config).

8. Related work

9. Conclusion

References